Sarthak Siddhpura
AU2240041
BTEP

Agentic Cloud Cluster

Table of Contents:

Brief about me:

I am a Computer Science and Engineering student at Ahmedabad University with a 3.46 CGPA. I have strong technical skills in C++, Go, and full-stack development, with experience in low-latency systems, Docker, and distributed computing. My interests lie in systems programming and in continuously exploring new technologies. As a Backend Engineer Intern at GoQuant Technologies Inc., I integrated multiple crypto exchanges, developed proxy solutions for critical integration challenges, and engineered a high-performance C++ trading system. I also have experience in teaching and leadership, serving as a Teaching Assistant in DSA and Applied Linear Algebra, and as the Competitive Programming Lead in the Programming Club for the academic year 2024-2025. I will be joining Concentric AI in Bangalore as an SWE intern from 2026.

About Agentic Cloud Cluster:

This project will be built on top of my earlier *Agentic Cloud Cluster* developed as a part of the CSE-540 Cloud Computing course. The system is a distributed cluster framework made in Go. It schedules and executes containerized workloads across different nodes in a heterogeneous environment. The framework also featured an agentic scheduler, which utilized machine learning-based estimation using real-time resource monitoring, past performance data of different worker nodes under previous loads, and their current load.

Organization to benefit from the project:

Cloud-based organizations are expected to benefit from this project, given the ongoing research in this domain. Cloud infrastructure relies on distributed clusters to support various workloads across different resources. The conventional approach in schedulers overlooks contention and the requirements of tasks. The previous ”Agentic Cloud Cluster” incorporates an intelligent scheduler for task categorization, regression for runtime computation, and risk-driven allocation.

Problem Statement:

A heuristic fails in a cluster with spikes or failures, resulting in an increased number of SLA violations. Common strategies, such as FCFS or rule-based scheduling, assume stable workloads and balanced resource usage. Thus, they perform poorly when the load is uneven, and task runtimes are unpredictable. Since some preset rules cannot adapt to sudden changes in the system conditions, a reinforcement learning approach is favourable as it can learn from past outcomes and adjust scheduling decisions dynamically under uncertainty.

Planned Deliverables:

RL scheduler to cut completion time by 15-25%, recovery via rescheduling. Modular RL framework, insights on heterogeneous efficacy for production.

- Less than 5% SLA violations in Dynamics.
- Integration of the RL module with the existing codebase of Agentic Cloud Cluster.
- Report on balancing/affinity gains.

Novelty:

This project extends the Agentic Cloud Cluster by integrating reinforcement learning for dynamic job scheduling and failure-aware rescheduling. The proposed work introduces an RL-based scheduler into an existing agentic cluster framework to overcome the limitations of any heuristic and rule-based scheduling approaches. The limitations arise when we are presented with workload variability, resource heterogeneity, and node failures. By learning from real-time cluster states and previous execution outcomes, the system will adapt to uncertain task runtimes and sudden load spikes, effectively closing the adaptability gaps present in conventional cloud schedulers, such as priority-based scheduling.

Resources:

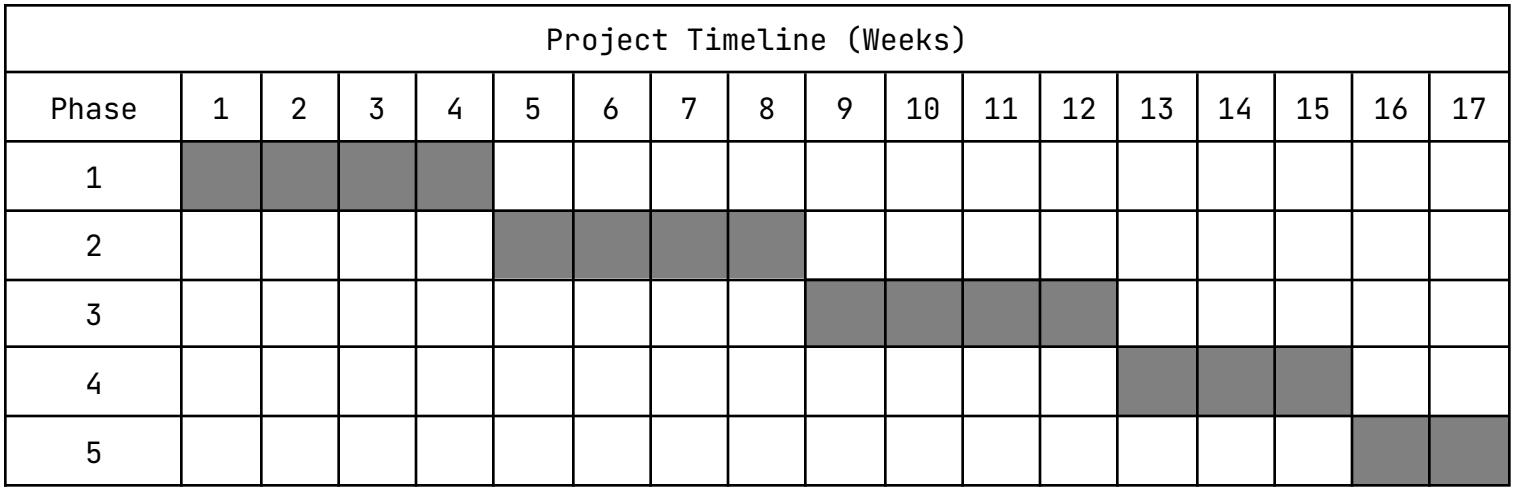
- Go for the entire framework and orchestrations
- Python (Gymnasium) for RL integration.
- Docker for containerized tasks
- gRPC for communications
- MongoDB for persistence
- Custom scripts/Jupyter notebooks for simulation and RL, along with Prometheus for evaluation and monitoring.

Development approaches:

1. Evaluate RL methods(DQN) and recovery techniques.
2. Preprocess data, define RL environment with states: loads/affinities, action: assign, reward: efficiency, and violations.
3. Train offline, fine-tune online.
4. Add rescheduling where heartbeat timeouts trigger RL replan/queue update.
5. Integrate benchmark baseline v/s extended.

Timeline and Gantt chart:

Phase	Duration	Activities
Phase 1	Week 1–4	RL scheduling literature and methodology
Phase 2	Week 5–8	RL setup, preprocessing, initial training
Phase 3	Week 9–12	RL/rescheduling integration; simulations
Phase 4	Week 13–15	Evaluation vs. baseline
Phase 5	Week 16–17	Report and presentation



Updates - Week 1

Identified a research survey paper [5] which provides the following:

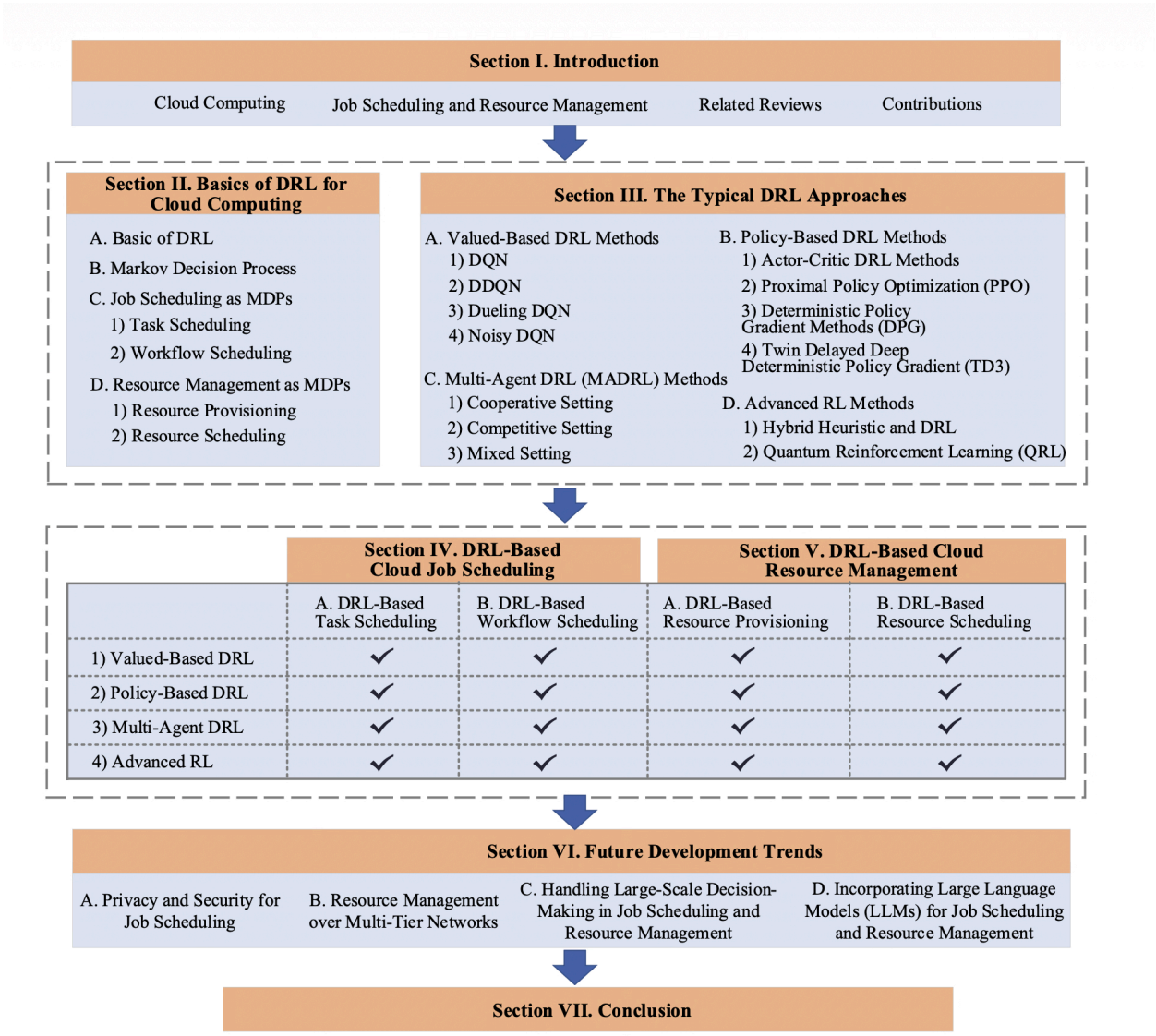


Fig. 1: The general architecture of this review

This will provide a robust foundation for the basis of reinforcement learning - understanding, implementation and comparison in the final project.

Definition of learning agent:

In reinforcement learning, the learning agent typically addresses sequential decision-making problems. The environment in with which the agent interacts is a Markov Decision Process (MDP) defined by the quintuple {S, A, T, R, γ}, where:

- S = state space
- A = action space
- T = transition probabilities from one state to another
- R = reward based on the agent's action on state s_t using action a_t
- γ = discount factor to govern the tradeoff between immediate and future rewards

The action taken a_t taken by the agent by observing the state s_t is given by the policy π which it follows. Thus:

$$\pi: S \rightarrow A$$

The goal of the agent will be to maximise the expected cumulative reward over time, which is given by:

$$\sum_{t=0}^{\infty} \gamma^{t} r_{t}(s_{t}, a_{t})$$

Our rewards can be like minimizing wait time in the queue, and maximizing resource utilization, as idle resources are more expensive.

Furthermore, we can have different types of environments based on the requirement as:

- Job Scheduling MDPs
    a. Task scheduling: limited to a single task
    b. Workflow scheduling: deals with tasks with dependencies, or a DAG (directed acyclic graph) of tasks

Our focus will be on the Task Scheduling variant of MDPs.

Why Deep Reinforcement Learning (DRL)?

- Traditional RL methods struggle with high-dimensional state spaces and complex decision problems, especially when state representations and decision-making processes are not straightforward.
- Thus, to address these challenges, we use DRL, which combines principles of RL with the powerful function approximation capabilities of deep neural networks.

Typical DRL algorithms include:

1. Value-based DRL models
    a. DQN
    b. DDQN
    c. Dueling DQN
    d. Noisy DQN
2. Policy-based DRL methods
    a. Actor Critic DRL methods
    b. Proximal Policy Optimization (PPO)
    c. Deterministic Policy Gradient Methods
    d. Twin Delayed Deterministic Policy Gradient
3. Multi-agent DRL methods
    a. Cooperative Setting
    b. Competitive Setting
    c. Mixed Setting
4. Advanced RL Methods
    a. Hybrid Heuristic and DRL
    b. Quantum Reinforcement Learning (QRL)

What does deep learning change?

In regular Q-functions, we maintain a table to store the Q-values or state-action values. In DRL, we will replace this table with a neural network. Problem solved in this manner: scalability. We cannot scale our Q-table in regular reinforcement learning. But the neural network will work as a functional approximator to return the desired Q-value.

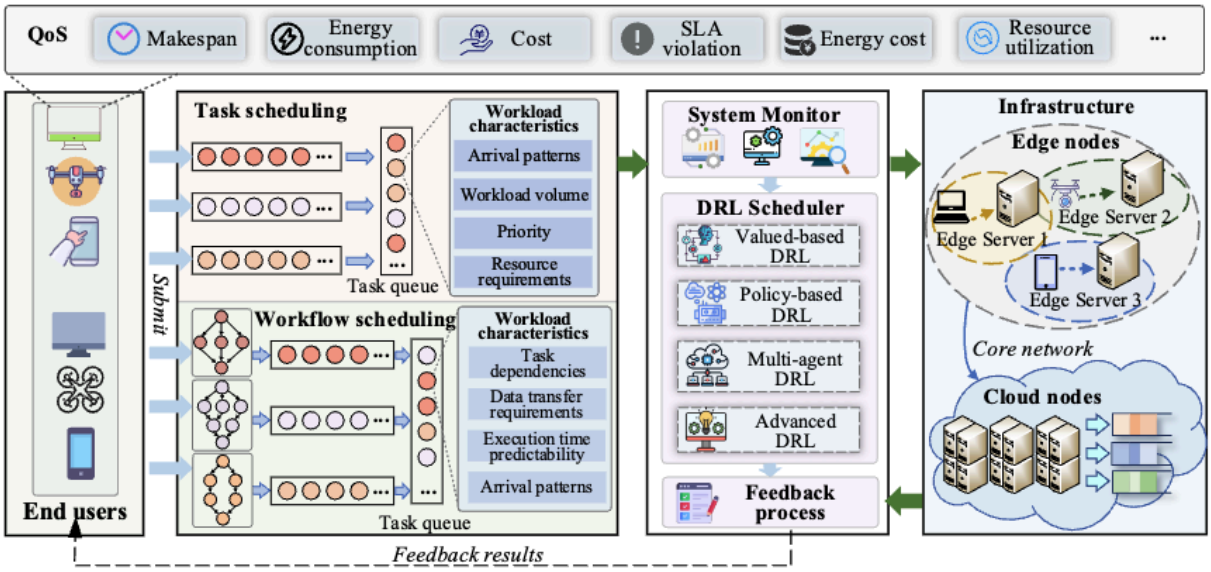


Fig. 3: The system architecture of job scheduling in cloud computing

Week 2 - 3

TABLE III: Summary of typical works on DRL-based workflow scheduling

Reference	Method	Others	Environment	Optimization objective			
				Makespan	Cost	Energy consumption	Others
[112]	Deep Q-learning	-	Cloud computing	✓	-	-	Load balance
[113]	DQN	-	Cloud computing	✓	-	-	Resource utilization
[114]	DQN	SA	Cloud computing	✓	✓	-	Task failures; communication costs
[115]	DQN	SA	Cloud computing	✓	✓	-	-
[116]	DQN	GA	Cloud computing	✓	✓	-	-
[117]	DDQN	-	Cloud computing	✓	-	-	Resource usage
[118]	DDQN	-	Cloud computing	✓	✓	-	-
[119]	D3QN	-	Cloud computing	✓	✓	-	Fairness and continuity
[120]	DQN	-	Cloud computing	✓	-	-	Load balance
[121]	DQN	-	Edge-Cloud collaboration	✓	-	-	-
[122]	DQN	-	Edge-Cloud collaboration	✓	-	-	VM utilization; failed tasks rate
[123]	DQN	-	Edge-Cloud collaboration	✓	-	-	Network throughput
[124]	DDQN	LSTM	Edge-Cloud collaboration	✓	-	✓	Utility
[125]	D3QN	-	Edge computing	✓	✓	✓	-
[126]	AC	-	Cloud computing	✓	-	-	-
[127]	AC	LSTM	Cloud computing	✓	-	-	-
[128]	AC	-	Cloud computing	-	-	-	Bounded slowdown; resource utilization
[129]	A2C	-	Cloud computing	-	-	✓	Carbon emission
[130]	PPO	GCN	Cloud computing	✓	-	-	Job latency
[131]	PPO	GCN	Cloud computing	✓	-	-	Resource utilization
[132]	PPO	Self-attention mechanism	Cloud computing	-	✓	-	-
[133]	DDPG	-	Cloud computing	✓	-	-	-
[134]	AC	-	Edge-Cloud collaboration	✓	-	✓	-
[135]	AC	GCN	Edge-Cloud collaboration	✓	-	-	Energy cost; network traffic; load balance
[136]	A3C	-	Edge-Cloud collaboration	-	-	-	Task rejection rate; resource utilization
[137]	PPO	GNN	Edge-Cloud collaboration	✓	-	-	Migration time; energy cost
[138]	PPO	-	Edge-Cloud collaboration	✓	✓	-	Load balance
[139]	PPO	-	Edge computing	✓	-	-	Resource utilization
[140]	DDPG	-	Edge computing	✓	-	-	-
[141]	MADRL	-	Cloud computing	-	✓	✓	Load balance; resource utilization
[142]	MADRL	-	Cloud computing	✓	✓	-	-
[143]	MADDPG	-	Cloud computing	✓	-	✓	-
[144]	MADRL	-	Cloud computing	✓	✓	-	Execution interruptions
[145]	MADRL	-	Edge computing	✓	-	✓	Success rate; resource utilization
[146]	MADRL	-	Edge computing	✓	-	✓	Success rate; load balance
[147]	MADRL	-	Edge computing	✓	-	-	-
[148]	Depth-First-Search Coalition RL	-	Cloud computing	✓	-	-	-
[149]	DQN	Transformer	Cloud computing	✓	✓	✓	-
[150]	DDQN	GNN	Cloud computing	✓	-	-	-
[151]	DQN	Primary backup	Edge computing	✓	-	-	-
[152]	DQN	QML	Edge-Cloud collaboration	✓	✓	-	Power utilization; violation of delay
[153]	DQN	Transformer	Edge-Cloud collaboration	✓	✓	✓	Weighted cost

Based on this chart, we can narrow down our research and target a particular deep reinforcement learning algorithm for our use case.

I believe the PPO algorithm will be the best fit in our use case as it is highlighted to be a superior choice for a dynamic cloud environment like our case. The paper explicitly states that Value-Based methods (like DQN) often struggle with "value overestimation" and suboptimal convergence in highly dynamic scenarios. PPO addresses this by stabilizing the training process, making it safer and more efficient for real-time.

The paper states that PPO algorithm is:

- used to simultaneously optimize metrics like operational cost, makespan, and resource utilization. We would definitely like to increase the resource utilization as idle resources are expensive!
- also noted for its adaptability to dynamic and unpredictable scheduling environments.

The review paper presents with multiple approaches of PPO implementations for different use cases under the following references under it:

1. Paper [86]:
   This paper integrates priority rules with PPO to handle random task arrivals.
2. Paper [88]:
   This paper uses PPO for real time workload shifting to minimize makespan and operational cost.
3. Paper [130] & [137]:
   If we implement a workflow scheduling mechanism where our master node executes and maintains a workflow which is a DAG of tasks, then the paper suggests moving with this approach.

Paper References:

1. [86] J. Jin and Y. Xu, “Optimal policy characterization enhanced proximal policy optimization for multitask scheduling in cloud computing,” IEEE Internet of Things Journal, vol. 9, no. 9, pp. 6418–6433, 2021.
2. [88] J. Zhao, M. A. Rodr´ ıguez, and R. Buyya, “A deep reinforcement learning approach to resource management in hybrid clouds harnessing renewable energy and task scheduling,” in 2021 IEEE 14th International Conference on Cloud Computing. IEEE, 2021, pp. 240–249.
3. [130] J. Xue, T. Wang, and P. Cai, “Towards efficient workflow scheduling over yarn cluster using deep reinforcement learning,” in 2023 IEEE Global Communications Conference, 2023, pp. 473–478.
4. [137] K. Zhu, Z. Zhang, S. Zeadally, and F. Sun, “Learning to Optimize Workflow Scheduling for an Edge–Cloud Computing Environment,”IEEE Transactions on Cloud Computing, 2024.

Here I found more documents that can help me better understand the PPO algorithm proposed in the review paper:

- https://spinningup.openai.com/en/latest/algorithms/ppo.html
- https://arxiv.org/abs/1707.06347

We will primarily use the first approach as that would be the best fit in our existing framework. However, I have found a linux-only utility - `criu` which can help me live-migrate a running container from one machine to another.

CRIU

https://criu.org/Main_Page

- Checkpoint Restore in userspace
- Allows to “freeze” a running application or entire container
- Save its current state to a set of files on a disk
- Later we can use those files to “restore” the application and pick up exactly where it left off.
- The size of backup stored on the disk is proportional to the memory consumption of the checkpoint.

The tradeoff in using this utility for live migration which we need to consider are:

- File IO:
  as this will include writing of files to store the backup
- NetworkIO:
  in the event, we need to send the backed up files to another machine

This implementation can enable us to effectively migrate containers from one machine to another that too from a checkpoint. For the snapshots we can have 2 options:

1. Take snapshots of the process at regular intervals.
   This approach is more fault tolerant, but it would require the workernodes to send the snapshots to master with the telemetry and heartbeat signals continuously which can increase the network IO.
2. Take the backup only when we need to migrate:
   This approach is disk efficient as we will not maintain the snapshots each and every time interval. However, if our workernode is down, then we cannot expect to “continue from where we left off”.

Integrating this utility will require significant changes in the existing framework, hence we can keep this for the future improvements for now.

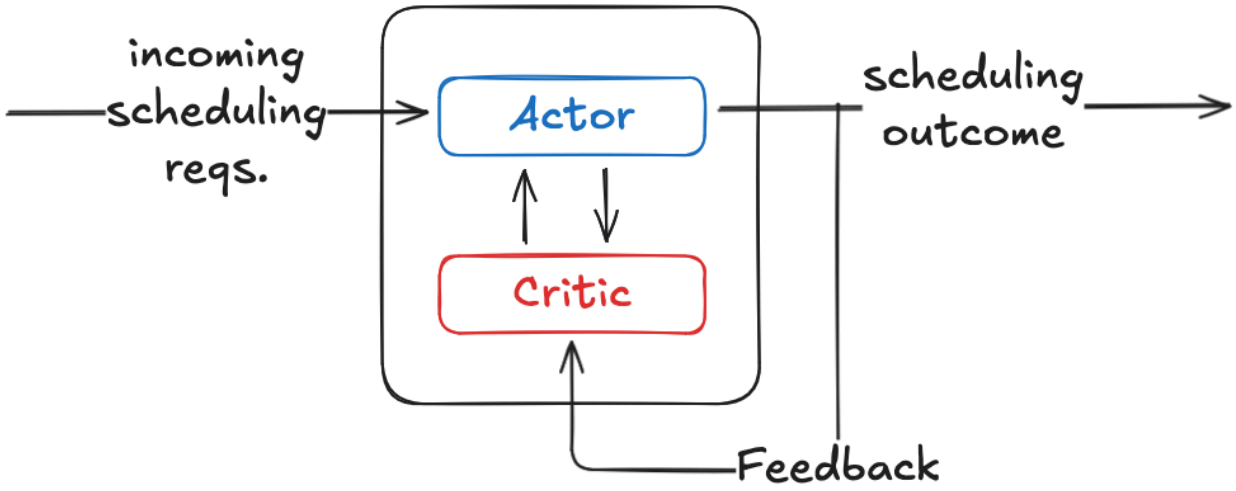
Week 4

Imp questions

1. Where to get the training data for the PPO algorithm ?
2. How would I define the state space for the PPO algo in our use case of DRL in scheduling ?
3. How and where do I “store” the trained model ?
4. Feedback loop for the live/online system ⇒ adapting online as the scheduler schedules.
5. Should I track the rescheduling requests with any provisions ?
6. After getting the scheduling outcome, should I consider the critic for ‘x’ number of times before actually scheduling or first schedule and then incorporate the critic ?

Ideally our model should handle the following:

- Incoming scheduling requests
- Learning at the same time ⇒ updating the weights and biases
- Persistence (we can use the existing mongodb)



The paper referenced as [86] provides the appropriate dataset for testing our framework

Updates from week 5

- Identified some vulnerabilities in the codebase.
- Add benchmarking in the codebase.
- Implemented PPO-based scheduling integrated with gRPC communication and MongoDB persistence.
- Explored SAC-CS (Soft Actor-Critic with Continuous State) algorithm and added supporting research references for comparative study.
- Hardened the scheduler by fixing task queueing issues and improving task cancellation handling.
- Enhanced runtime metadata tracking for better observability and debugging.
- Implemented task queue persistence and restoration mechanisms across the cluster.
- Added benchmarking utilities to support performance evaluation of scheduling strategies.
- Extended macOS worker support, enabling compatibility with Apple Silicon environments.
- Updated and refined literature survey documentation in the repository.
- Improved developer setup with dependency installation scripts and configuration cleanups (.gitignore, tooling support).

References:

1. Kubernetes Documentation. (n.d.). Kubernetes Architecture. Retrieved from https://kubernetes.io/docs/concepts/architecture/
2. Red Hat, Inc. (n.d.). What is KVM (Kernel-based Virtual Machine)?. Retrieved from https://www.redhat.com/en/topics/virtualization/what-is-KVM
3. AlgoMaster Blog. (n.d.). Design a Distributed Job Scheduler. Retrieved from http://blog.algomaster.io/p/design-a-distributed-job-scheduler
4. Chaudhry, A., et al. (2025). Murakkab: Resource-Efficient Agentic Workflow Orchestration in Cloud Platforms. arXiv. Retrieved from https://arxiv.org/pdf/2508.18298v1
5. Gu, Y., Liu, Z., Dai, S., Liu, C., Wang, Y., Wang, S., Theodoropoulos, G., & Cheng, L. (2025, January 2). Deep Reinforcement Learning for job scheduling and Resource Management in Cloud Computing: An Algorithm-Level Review. arXiv.org. https://arxiv.org/abs/2501.01007
6. CRIU. (n.d.). https://criu.org/Main_Page
7. Li, Z., Zhang, S., Li, Y., Liu, X., Huang, J., & Hu, J. (2025). Energy-Efficient container scheduling based on deep reinforcement learning in data centers. Computers, 14(12), 560. https://doi.org/10.3390/computers14120560